

Scalable Document Processing and Analysis Pipeline on Kubernetes

Project Deliverable 1

PREPARED BY

Taylor Bellegue

INSTITUTION

West Chester University

Chapter 1: Vision

Scalable Document Processing and Analysis Pipeline on Kubernetes

1.1 Project Vision

Organizations routinely need to process text-based information before it can be stored, analyzed, or used by other applications. Processing one document is straightforward, but a system that receives multiple requests must also address availability, scalability, reliability, and data retention. A cloud-native architecture provides a practical way to separate these responsibilities while keeping each component focused and manageable.

The vision for this project is to design and implement a scalable document processing and analysis pipeline on Kubernetes. A user will submit a small text document through a single application entry point. The document will then move through independent services for preprocessing, analysis, and storage. The analysis stage will produce useful document characteristics such as word count, sentence count, unique-word count, frequently occurring words, and basic sentiment. The completed result will be stored so that it can be retrieved later.

The project is intended to demonstrate how a distributed application can be deployed and operated rather than to create a complex natural language processing model. The document workflow gives each Kubernetes component a clear purpose while keeping the underlying application logic understandable. This allows the project to concentrate on container orchestration, service-to-service communication, horizontal scaling, self-healing, persistent storage, health monitoring, and resource management.

Kubernetes is central to the vision because each primary processing stage can be containerized and managed independently. A problem in one service should not require the entire application to be rebuilt or redeployed. Likewise, additional processing capacity can be added to the analysis stage without duplicating every component. The resulting system will illustrate how Kubernetes supports a modular application whose components can communicate, recover, scale, and preserve data within a unified platform.

Project focus: use an understandable document-processing workflow to demonstrate Kubernetes architecture and operations.

1.2 High-Level Architecture

At a high level, the proposed system follows a sequential pipeline. Each stage accepts the output of the previous stage and performs one clearly defined responsibility before passing the request forward.

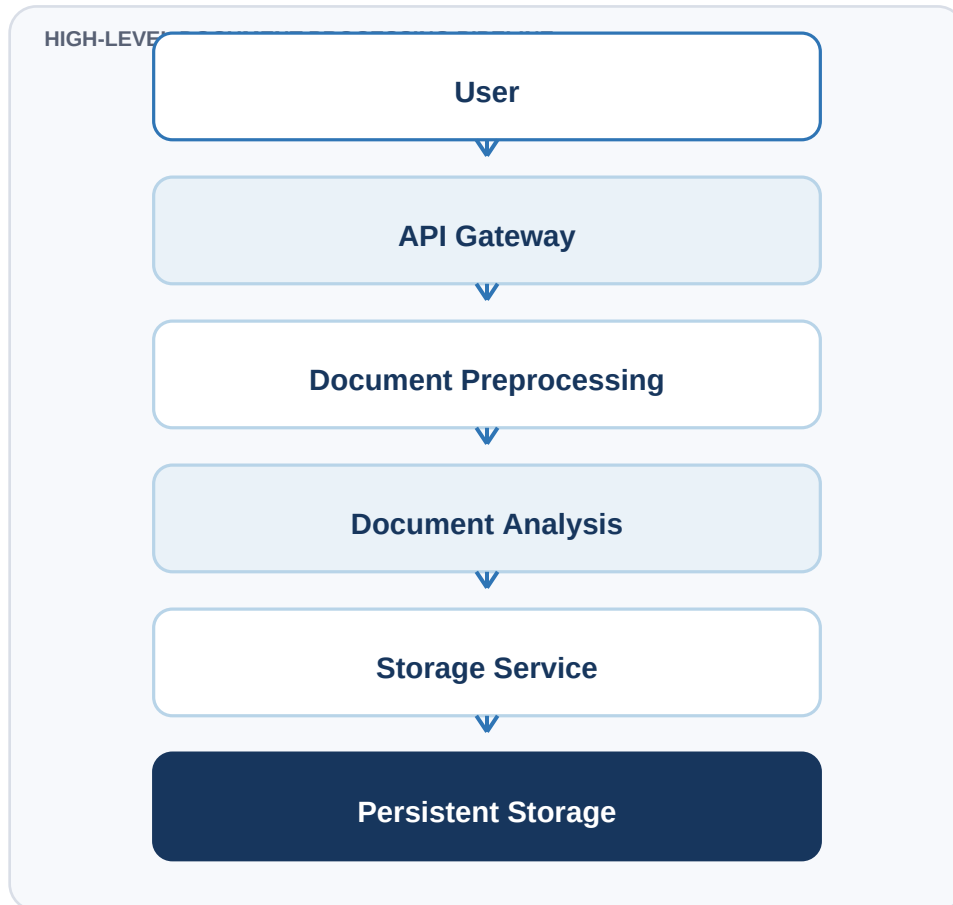


Figure 1. High-level architecture of the document processing and analysis pipeline.

The user interacts with the system through the API Gateway, which acts as the single entry point for document submission and result retrieval. The gateway forwards submitted text to the Document Preprocessing component, where the content is cleaned and normalized. The prepared text then moves to Document Analysis, which generates the planned document statistics and classification results. The Storage Service records the original document, processed content, and analysis output in Persistent Storage. This separation of responsibilities creates a clear workflow while allowing the major components to be deployed, managed, and scaled independently within Kubernetes.

The architecture is intentionally presented at a 10,000-foot level in this chapter. Chapter 2 will translate this vision into a proposed Kubernetes implementation, including component deployment, internal communication, the multi-container Pod design, scaling, resilience, and persistent storage.

Chapter 2: Proposed Implementation

Scalable Document Processing and Analysis Pipeline on Kubernetes

2.1 Proposed Application Components

The system will be implemented as four lightweight Python services using FastAPI and deployed as Kubernetes Deployments. Separating the workflow into services provides distinct stages for orchestration while keeping the application logic small enough for the project to remain focused on Kubernetes.

API Gateway

The API Gateway will be the single external entry point. It will accept small plain-text documents, assign each submission a document identifier, initiate processing, and return previously stored results.

Document Preprocessing Service

The preprocessing service will clean and normalize submitted text by trimming unnecessary whitespace, normalizing line breaks, handling selected punctuation, and validating that the document is not empty.

Document Analysis Service

The analysis service will calculate character count, word count, sentence count, unique-word count, frequently occurring words, and a lightweight positive, neutral, or negative sentiment result. It will be the primary component used to demonstrate scaling.

Storage Service

The storage service will save completed analyses as JSON files and retrieve them by document identifier. This file-based design avoids adding a database while still providing a clear use case for persistent storage.

2.2 Kubernetes Architecture

Each application component will run as a Kubernetes Deployment. Internal communication will use ClusterIP Services, while only the API Gateway will be exposed externally. Components will call stable Kubernetes Service names rather than individual Pod IP addresses, allowing Pods to be replaced without changing the application configuration.

Component	Service	Replicas	Primary Role
API Gateway	External	1	Submit and retrieve documents
Preprocessing	ClusterIP	1	Clean and validate text
Analysis	ClusterIP	3	Analyze document content
Storage	ClusterIP	1	Store and retrieve JSON results

At a high level, requests will flow from the API Gateway to preprocessing, then to analysis, then to storage. The Storage Service will mount a PersistentVolumeClaim for saved results. This follows the architecture introduced in Chapter 1 while adding the Kubernetes resources needed to implement it.

2.3 Multi-Container Pod, Scaling, and Resilience

The Analysis Deployment will provide the required multi-container Pod. Each analysis Pod will contain the analyzer and a logging sidecar. The analyzer will write processing logs to a shared emptyDir volume, while the sidecar will read that shared log and make the output available through Kubernetes logging. The shared volume is temporary and will not be used for result persistence.

The Analysis Deployment will initially run three replicas. Its ClusterIP Service will provide a single internal address while Kubernetes distributes requests among the available Pods. Scaling will be demonstrated by changing the replica count and verifying the running Pods.

Self-healing will be demonstrated by manually deleting one analysis Pod. Because the Deployment maintains a desired replica count of three, Kubernetes should create a replacement. The test will document the deletion and replacement with kubectl output or screenshots.

2.4 Persistence and Operational Configuration

The Storage Service will use a PersistentVolume and PersistentVolumeClaim mounted at a directory such as /data/results. Completed analyses will be written to that persistent location as JSON files. Persistence will be verified by saving results, replacing the Storage Pod, and confirming that the same results remain accessible afterward.

Each FastAPI service will also expose a /health endpoint for Kubernetes readiness and liveness probes. Lightweight CPU and memory requests and limits will be configured for the application containers, particularly the analyzer. Non-sensitive settings, such as internal service addresses and storage paths, may be supplied through ConfigMaps. If API authentication is included, the external gateway will use a simple key stored in a Kubernetes Secret rather than embedding it in source code.

2.5 Testing and Implementation Plan

Implementation will proceed incrementally. The four FastAPI services will first be developed and tested individually, then containerized, deployed to Kubernetes, and connected through Services. After the core pipeline operates successfully, the multi-container analysis Pod, scaling, self-healing, persistence, health probes, and resource settings will be added.

The primary test cases will include:

- A valid text document completes the full preprocessing, analysis, storage, and retrieval workflow.
- An empty or unsupported submission is rejected cleanly.
- Three analysis replicas can run behind one Kubernetes Service.
- A deleted analysis Pod is automatically replaced.
- Stored results remain available after the Storage Pod is replaced.
- Readiness and liveness endpoints report the health of application services.

All application source code, Dockerfiles, Kubernetes YAML manifests, testing commands, and project documentation will be maintained in the public GitHub repository. The manifests will be organized so that the application can be reproducibly deployed and removed as infrastructure as code.

2.6 Expected Outcome

The completed project is expected to provide a reproducible Kubernetes pipeline that accepts a text document, preprocesses and analyzes its contents, stores the result persistently, and allows the result to be retrieved later. More importantly, the project will demonstrate the course concepts of multiple Pods, a multi-container Pod, Kubernetes Services, horizontal scaling, self-healing, persistent storage, health monitoring, and declarative deployment. The document-analysis logic will intentionally remain lightweight so that the principal technical effort stays focused on Kubernetes integration and operation.